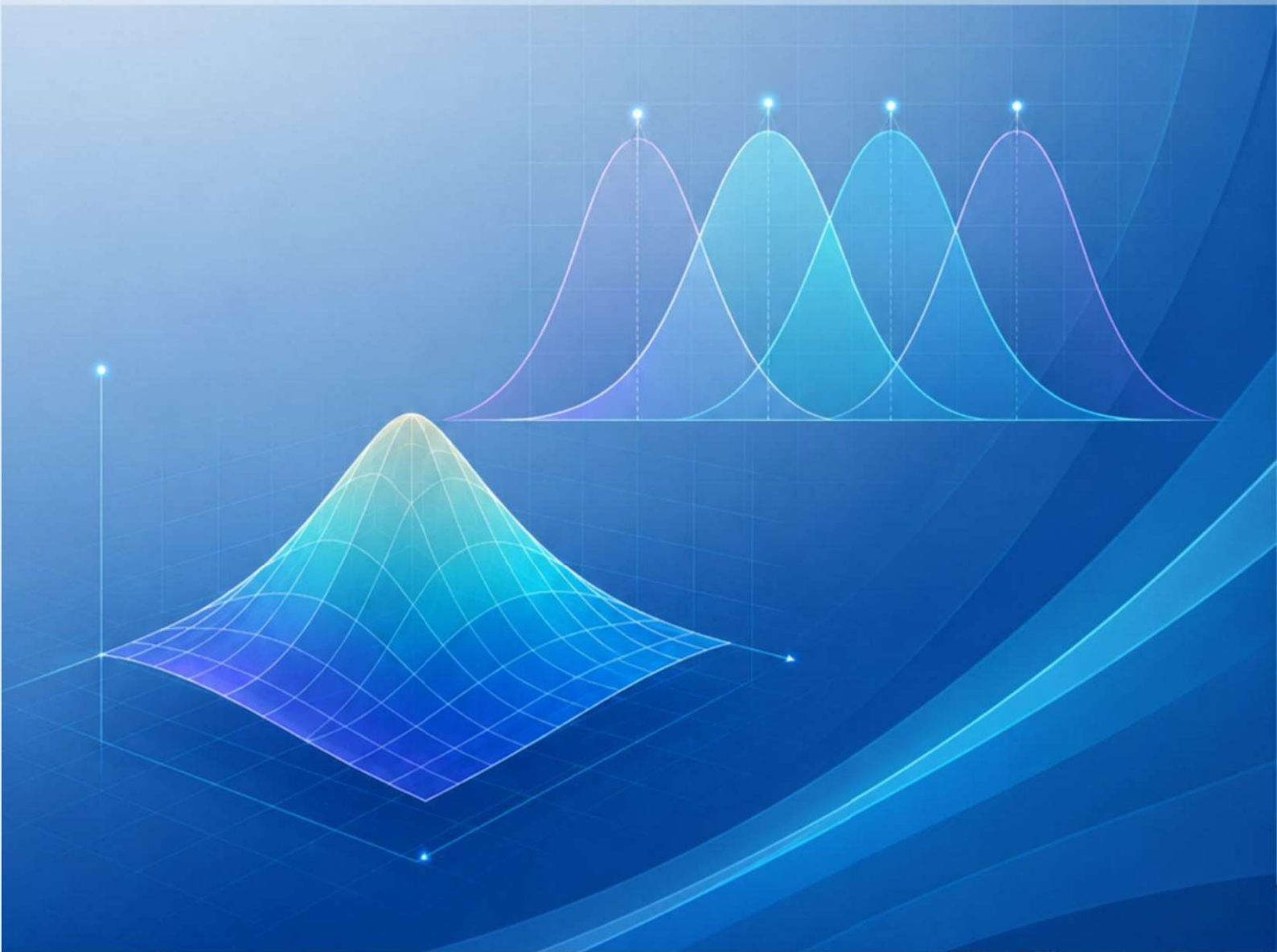




# MODUL PRAKTIKUM KONTROL CERDAS

**Disusun Oleh:**  
Dr. I Nyoman Kusuma Wardana

## MODUL 1

# Pemrograman Logika Fuzzy Menggunakan Script MATLAB

### Pendahuluan

Perancangan sistem fuzzy dapat dilakukan menggunakan **Fuzzy Logic Designer** pada MATLAB. Namun, selain menggunakan tampilan grafis, mahasiswa juga perlu memahami cara membangun sistem fuzzy melalui **pemrograman**. Pendekatan ini penting karena memberikan fleksibilitas yang lebih besar dalam mendefinisikan input, output, membership function, rule base, serta proses evaluasi sistem fuzzy.

Pada praktikum ini, mahasiswa akan mempelajari cara mendesain kontrol fuzzy menggunakan script MATLAB. Mahasiswa akan membuat sistem fuzzy secara bertahap, mulai dari menentukan variabel input dan output, menyusun fungsi keanggotaan, menambahkan aturan fuzzy, hingga menguji hasil inferensi fuzzy terhadap beberapa kondisi input. Dengan pendekatan pemrograman, setiap bagian dari sistem fuzzy dapat diamati dan dimodifikasi secara lebih terstruktur.

Praktikum ini juga menjadi dasar penting sebelum mahasiswa menerapkan kontrol fuzzy pada sistem yang lebih kompleks, seperti pengendalian level tangki di Simulink. Melalui kegiatan ini, mahasiswa diharapkan tidak hanya mampu menggunakan fitur bawaan MATLAB, tetapi juga memahami logika di balik proses pembentukan sistem fuzzy secara mandiri melalui kode program.

### Tujuan

Setelah menyelesaikan praktikum ini, Mahasiswa diharapkan mampu:

1. Memahami konsep dasar perancangan sistem fuzzy menggunakan pemrograman MATLAB.
2. Membuat sistem fuzzy melalui script MATLAB tanpa menggunakan Fuzzy Logic Designer.
3. Mendesain variabel input, output, membership function, dan rule base pada sistem fuzzy.
4. Melakukan evaluasi dan analisis keluaran sistem fuzzy terhadap beberapa variasi input.

### Alat dan Bahan

Alat yang dibutuhkan diperlihatkan pada Tabel 1.

Tabel 1. Kebutuhan praktikum

| No | Nama Alat dan Bahan | Jumlah | Keterangan          |
|----|---------------------|--------|---------------------|
| 1  | MATLAB/Simulink     | 1      | Desain model tangki |



## Langkah-langkah Percobaan

### 1. Amati kembali kasus “Dinner for Two”. Berikut kasusnya:

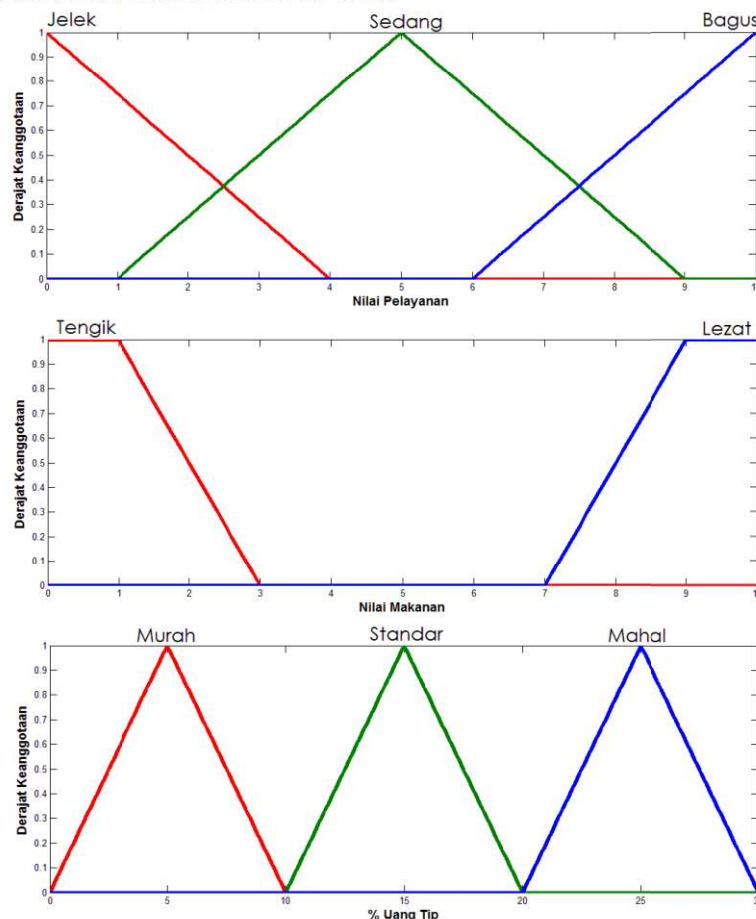
Karena hari ini Valentine, Romi ingin mengajak Juli *dinner* di suatu restoran. Sebelum berangkat *dinner*, Romi berpikir akan membagi kebahagiaannya malam ini kepada pelayan restoran dengan memberikan uang **tip**. Dia akan memberikan uang tip sebesar **5-25%** dari total belanjanya. Besarnya uang tip akan dilihat dari tingkat **pelayanan** (*service*) dan kualitas **makanan** (*food*) yg dihidangkan.

Adapun aturan pemberian tip yg ditetapkan oleh Romi adalah sebagai berikut:

- Jika **Pelayanan Jelek ATAU Makanan Tengik**, maka **Tip Murah**
- Jika **Pelayanan Sedang**, maka **Tip Standar**
- Jika **Pelayanan Bagus ATAU Makanan Lezat**, maka **Tip Mahal**

Berapa **Tip** yang harus diberikan jika penilaian terhadap **pelayanan** = 7 dan **makanan** = 8?

### 2. Amati fungsi keanggotaan untuk input dan output. Berikut detail desain fungsi keanggotaan untuk kasus dinner for two:



### 3. Lakukan Pembuatan Script MATLAB. Pada jendela utama MATLAB, pilih New Script. Ikutilah langkah-langkah berikut:

Pertama, kita akan membuat obyek fuzzy kita. Kita pilih mamfis untuk tipe Mamdani dan beri nama "tipper".

```
% buat Mamdani FIS
fis = mamfis(Name="tipper");
```

Selanjutnya, tambahkan variabel input, yaitu Pelayanan dan Makanan. Lengkapi juga dengan rentang untuk tiap-tiap variabel.

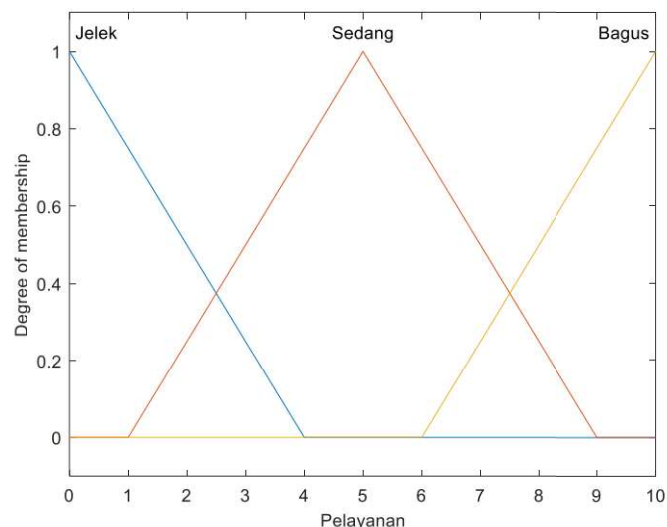
```
% tambahkan variabel input
fis = addInput(fis, [0 10], Name="Pelayanan");
fis = addInput(fis, [0 10], Name="Makanan");
```

Sekarang, tambahkan membership function (MF). Kita mulai menambahkan MF untuk variabel Pelayanan. Masukkan jenis MF, range, dan nama tiap-tiap MF.

```
% tambahkan membership function ke input "Pelayanan"
fis = addMF(fis, "Pelayanan", "trimf", [-4 0 4], Name="Jelek");
fis = addMF(fis, "Pelayanan", "trimf", [1 5 9], Name="Sedang");
fis = addMF(fis, "Pelayanan", "trimf", [6 10 14], Name="Bagus");
```

Plot untuk melihat bagaimana input Pelayanan terbentuk.

```
% plot MF "Pelayanan"
figure; plotmf(fis, "input", 1)
```



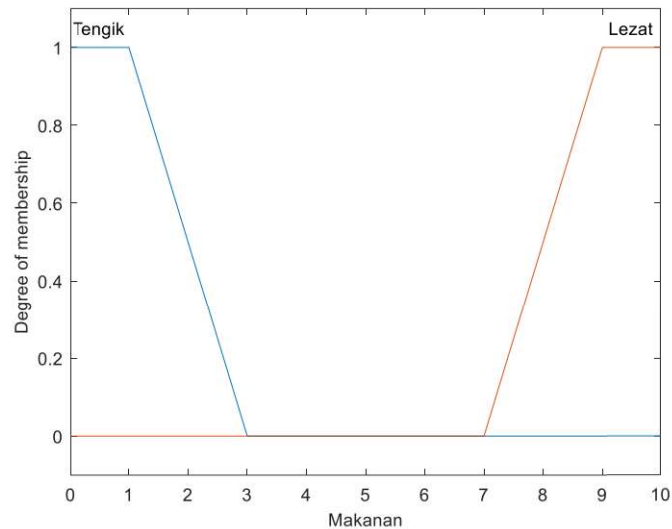
Lanjut ke variabel Makanan. Tambahkan MF untuk Makanan.

```
% tambahkan membership function ke input "Makanan"
fis = addMF(fis, "Makanan", "trapmf", [0 0 1 3], Name="Tengik");
fis = addMF(fis, "Makanan", "trapmf", [7 9 10 10], Name="Lezat");
```

Plot input Makanan.

```
% plot MF "Makanan"
figure; plotmf(fis, "input", 2)
```





Setelah kedua input selesai, langkah selanjutnya adalah membentuk output. Tambahkan variabel output, yaitu Tip.

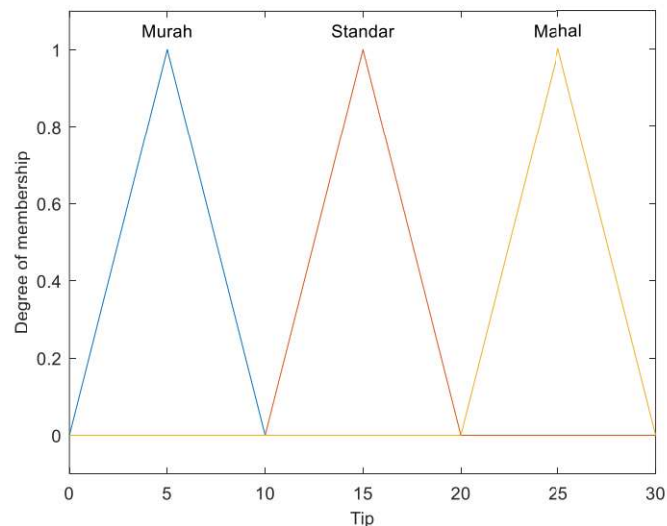
```
% tambahkan variabel output
fis = addOutput(fis,[0 30],Name="Tip");
```

Tambahkan MF, range, dan nama tiap-tiap MF pada output.

```
% tambahkan membership function ke output "Tip"
fis = addMF(fis,"Tip","trimf",[0 5 10],Name="Murah");
fis = addMF(fis,"Tip","trimf",[10 15 20],Name="Standar");
fis = addMF(fis,"Tip","trimf",[20 25 30],Name="Mahal");
```

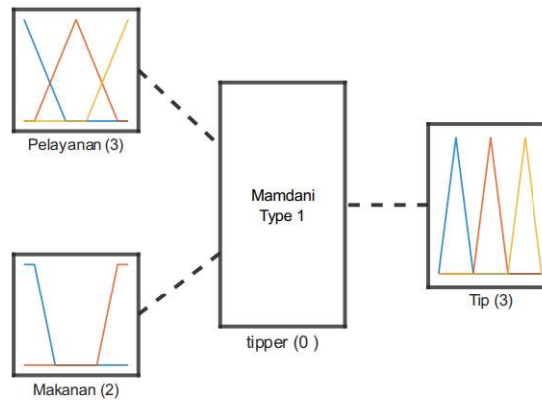
Tampilkan plot untuk variabel output.

```
% plot MF "Tip"
figure; plotmf(fis, "output",1)
```



Kita juga dapat menampilkan sistem fuzzy secara keseluruhan.

```
% tampilkan sistem fuzzy
figure; plotfis(fis)
```



System tipper: 2 inputs, 1 outputs, 0 rules

Ketika Input dan Output telah terbentuk, maka langkah selanjutnya adalah membentuk aturan (*rules*).

```
% Jika Pelayanan Jelek ATAU Makanan Tengik, maka Tip Murah
aturan1 = [1 1 1 1 2];
% Jika Pelayanan Sedang, maka Tip Standar
aturan2 = [2 0 2 1 1];
% Jika Pelayanan Bagus ATAU Makanan Lezat, maka Tip Mahal
aturan3 = [3 2 3 1 2];
```

#### Keterangan:

- kolom 1** = indeks MF untuk input pertama
- kolom 2** = indeks MF untuk input kedua
- kolom 3** = indeks MF untuk output
- kolom 4** = rule weight (0-1)
- kolom 5** = operator (1=AND, 2=OR)

Ketika matriks tiap-tiap aturan telah terbentuk, maka langkah selanjutnya adalah mengkombinasikan semua aturan tersebut dan memasukkan ke sistem FIS kita. Gunakan metode penggabungan matriks pada MATLAB. Gunakan titik koma (;) untuk memisahkan tiap aturan.

```
% Kombinasikan aturan
listAturan = [aturan1; aturan2; aturan3];

% tambahkan aturan ke FIS
fis = addRule(fis, listAturan);
```

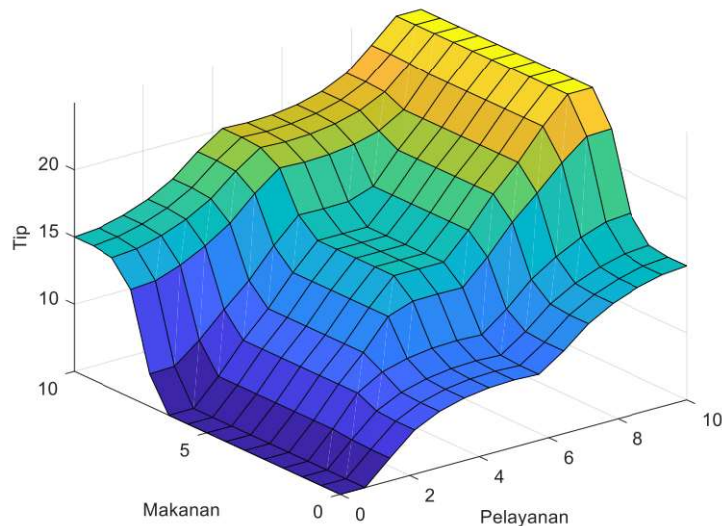
Kita dapat melihat aturan yang terbentuk. Pastikan bahwa semua aturan telah benar.

```
% tampilkan aturan
showrule(fis)
```

```
ans =
3x73 char array
'1. If (Pelayanan is Jelek) or (Makanan is Tengik) then (Tip is Murah) (1) '
'2. If (Pelayanan is Sedang) then (Tip is Standar) (1) '
'3. If (Pelayanan is Bagus) or (Makanan is Lezat) then (Tip is Mahal) (1) '
```

Kita juga dapat menampilkan control surface yang terbentuk

```
% tampilkan control surface  
figure; gensurf(fis)
```



Terakhir, lakukan evaluasi terhadap model fuzzy kita.

```
% evaluasi FIS (Pelayanan = 7, Makanan = 8)  
evalfis(fis,[7 8])
```

```
ans =
```

```
19.9980
```

4. **Simpan Script MATLAB.** Jangan lupa menyimpan skrip yang telah dibuat. Simpan dalam file berekstensi **.m** dan tanpa spasi, misalnya **desainFuzzyTip.m**.

## Tugas Peserta Praktikum

### Tugas 1

- Amatilah studi kasus berikut:

Sebuah sistem AC otomatis dirancang untuk mengatur **daya pendingin** berdasarkan **suhu ruangan** dan **jumlah orang** di dalam ruangan. Semakin tinggi suhu dan semakin banyak orang, maka daya AC yang dibutuhkan juga semakin besar. Sebaliknya, jika suhu ruangan rendah dan jumlah orang sedikit, daya AC dapat dibuat lebih rendah agar lebih hemat energi. Dengan logika fuzzy, kondisi suhu dan jumlah orang dibagi ke dalam beberapa kategori untuk menentukan keluaran berupa **daya AC** secara lebih fleksibel dan adaptif. Berikut keterangan Input dan Output kontroler:

**Input 1: SuhuRuangan (°C): [18 35]**



- **rendah:** trapmf [18 18 20 23]
- **sedang:** trimf [22 26 30]
- **tinggi:** trapmf [28 32 35 35]

**Input 2: JumlahOrang: [0 10]**

- **sedikit:** trapmf [0 0 2 4]
- **sedang:** trimf [3 5 7]
- **banyak:** trapmf [6 8 10 10]

**Output: DayaAC (%): [0 100]**

- **rendah:** trapmf [0 0 20 40]
- **sedang:** trimf [30 50 70]
- **tinggi:** trapmf [60 80 100 100]

Rules ditentukan sebagai berikut (gunakan konektor **AND**):

|             |        | JumlahOrang |        |        |
|-------------|--------|-------------|--------|--------|
|             |        | Sedikit     | Sedang | Banyak |
| SuhuRuangan | Rendah | Rendah      | Rendah | Sedang |
|             | Sedang | Sedang      | Sedang | Tinggi |
|             | Tinggi | Tinggi      | Tinggi | Tinggi |

- Desainlah kontroler fuzzy berdasarkan kasus tersebut dengan **Fuzzy Logic Designer**.
- Sekarang, buatlah kontroler yang sama namun menggunakan script pemrograman MATLAB.
- Bandingkan hasil antara menggunakan Fuzzy Logic Designer dan script MATLAB menggunakan Tabel 1 berikut:

Tabel 1. Perbandingan Fuzzy Logic Designer (FLD) dan Skrip MATLAB

| No | SuhuRuangan | JumlahOrang | DayaAC |       |
|----|-------------|-------------|--------|-------|
|    |             |             | FLD    | Skrip |
| 1  | 23          | 10          | ...    | ...   |
| 2  | 20          | 3           | ...    | ...   |
| 3  | 25          | 5           | ...    | ...   |
| 4  | 30          | 8           | ...    | ...   |
| 5  | 28          | 2           | ...    | ...   |
| 6  | 22          | 1           | ...    | ...   |
| 7  | 33          | 7           | ...    | ...   |

## Tugas 2

- Amatilah studi kasus berikut:

Suatu ruangan pertemuan menggunakan beberapa kipas untuk mengkondisikan udara dalam ruangan. Parameter input adalah **Suhu [20-40]** dan **Kelembapan [20-90]**. Parameter yang dikontrol adalah level **Kecepatan kipas [0-100%]**.

Berikut keterangan Input dan Output kontroler:

**Input 1: Suhu (°C): [20 40]**

- **dingin:** trapmf [20 20 24 26]
- **hangat:** trimf [25 28.5 32]
- **panas:** trapmf [31 34 40 40]

**Input 2: Kelembapan (%): [20 90]**

- **rendah:** trapmf [20 20 35 45]
- **sedang:** trimf [40 55 70]
- **tinggi:** trapmf [65 75 90 90]

**Output: KecepatanKipas (%): [0 100]**

- **rendah:** trapmf [0 0 20 40]
- **sedang:** trimf [30 50 70]
- **tinggi:** trapmf [60 80 100 100]

Rules ditentukan sebagai berikut (konektor AND):

|      |        | Kelembapan |        |        |
|------|--------|------------|--------|--------|
|      |        | Rendah     | Sedang | Tinggi |
| Suhu | Dingin | Rendah     | Rendah | Sedang |
|      | Hangat | Rendah     | Sedang | Tinggi |
|      | Panas  | Sedang     | Tinggi | Tinggi |
|      |        |            |        |        |

- Desainlah kontroler fuzzy berdasarkan kasus tersebut dengan **Fuzzy Logic Designer**.
- Sekarang, buatlah kontroler yang sama namun menggunakan script pemrograman MATLAB.
- Bandingkan hasil antara menggunakan Fuzzy Logic Designer dan script MATLAB menggunakan Tabel 2 berikut:

Tabel 2. Perbandingan Fuzzy Logic Designer (FLD) dan Skrip MATLAB

| No | Suhu | Kelembapan | KecepatanKipas |       |
|----|------|------------|----------------|-------|
|    |      |            | FLD            | Skrip |
| 1  | 27   | 20         | ...            | ...   |
| 2  | 20   | 60         | ...            | ...   |
| 3  | 25   | 55         | ...            | ...   |
| 4  | 30   | 78         | ...            | ...   |
| 5  | 28   | 50         | ...            | ...   |
| 6  | 22   | 90         | ...            | ...   |
| 7  | 39   | 45         | ...            | ...   |



## MODUL 2

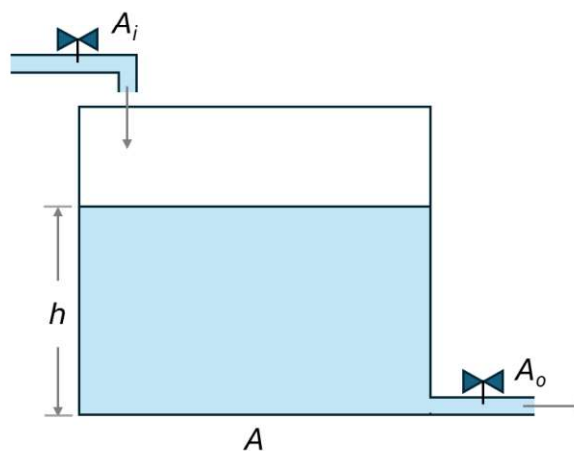
# Desain Kontrol Fuzzy untuk Pengendalian Level Air pada Model Tangki Sederhana Menggunakan MATLAB/Simulink

### Pendahuluan

Pengendalian level cairan merupakan salah satu kasus dasar yang banyak dijumpai dalam sistem kendali proses. Sistem ini dapat ditemukan pada berbagai aplikasi industri, seperti tangki penyimpanan air, tangki bahan kimia, sistem irigasi, pengolahan limbah, hingga proses pencampuran fluida. Meskipun terlihat sederhana, pengendalian level cairan memiliki karakteristik dinamis yang penting untuk dipahami, karena perubahan debit masuk dan debit keluar akan memengaruhi tinggi permukaan cairan terhadap waktu.

Pada praktikum ini, mahasiswa akan mempelajari sistem satu tangki sederhana sebagai objek kendali. Prinsip dasar yang digunakan dalam pemodelan sistem tangki adalah hukum kekekalan massa, yaitu laju akumulasi massa di dalam tangki sama dengan laju massa yang masuk dikurangi laju massa yang keluar. Dengan mengasumsikan massa jenis cairan konstan, hubungan tersebut dapat dinyatakan dalam bentuk perubahan ketinggian cairan terhadap waktu. Model matematis inilah yang kemudian digunakan sebagai dasar untuk membangun plant tangki di MATLAB/Simulink.

Secara umum, dinamika level air pada tangki dipengaruhi oleh luas penampang tangki, debit air masuk, debit air keluar, serta pengaruh gravitasi pada aliran keluar. Debit masuk biasanya dapat diatur melalui bukaan valve atau pompa, sedangkan debit keluar dapat bergantung pada tinggi permukaan air. Oleh karena itu, sistem tangki sederhana dapat menjadi contoh yang baik untuk memahami hubungan antara model matematis, simulasi sistem dinamis, dan perancangan sistem kendali. Sistem tangki sederhana diperlihatkan pada Gambar 1.



**Gambar 1. Sistem tangki sederhana**

Model level cairan pada Gambar 1 dapat dijabarkan sebagai berikut:

$$\frac{dh}{dt} = \frac{A_i V_i - A_o \sqrt{2gh}}{A} \quad (1)$$

Dari Persamaan (1), mahasiswa diminta membangun model tangki di Simulink, dan selanjutnya merancang kontroler fuzzy menggunakan Fuzzy Logic Designer pada MATLAB. Kontroler fuzzy digunakan untuk mengatur debit masuk agar level air dapat mengikuti nilai setpoint yang diinginkan. Melalui praktikum ini, mahasiswa diharapkan mampu menghubungkan konsep dasar pemodelan dinamika sistem dengan implementasi kontrol cerdas. Mahasiswa juga akan memperoleh pengalaman langsung dalam membangun *plant* di Simulink, mendesain *membership function*, menyusun *rule base* fuzzy, mengimpor sistem fuzzy ke Simulink, serta menganalisis respons sistem terhadap perubahan *setpoint*. Praktikum ini menjadi langkah awal untuk memahami penerapan kontrol fuzzy pada sistem proses yang lebih kompleks.

## Tujuan

Setelah menyelesaikan praktikum ini, Mahasiswa diharapkan mampu:

1. Memodelkan dinamika level air pada satu tangki sederhana di Simulink.
2. Mengamati respons sistem tanpa kontrol.
3. Mendesain kontroler fuzzy menggunakan Fuzzy Logic Designer MATLAB.
4. Mengimpor sistem fuzzy ke Simulink.
5. Menganalisis performa kontrol fuzzy berdasarkan respons level air terhadap setpoint.

## Alat dan Bahan

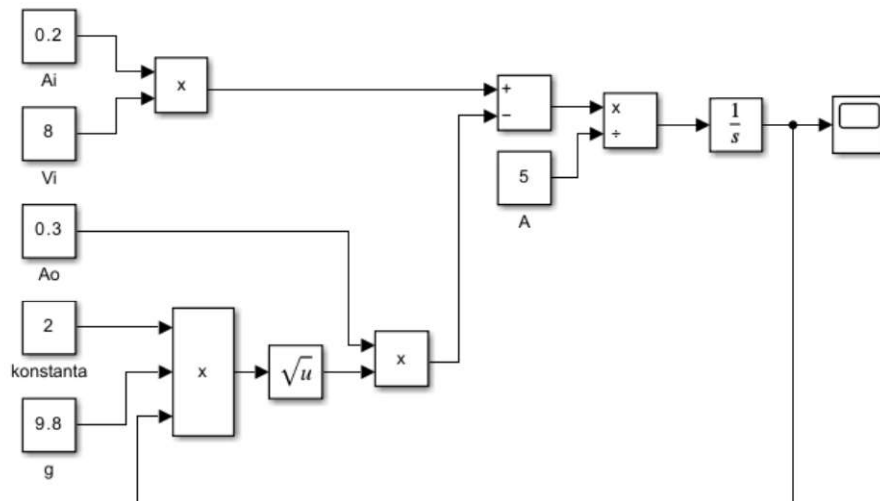
Alat yang dibutuhkan diperlihatkan pada Tabel 1.

Tabel 1. Kebutuhan praktikum

| No | Nama Alat dan Bahan  | Jumlah | Keterangan            |
|----|----------------------|--------|-----------------------|
| 1  | MATLAB/Simulink      | 1      | Desain model tangki   |
| 2  | Fuzzy Logic Designer | 1      | Desain sistem kontrol |

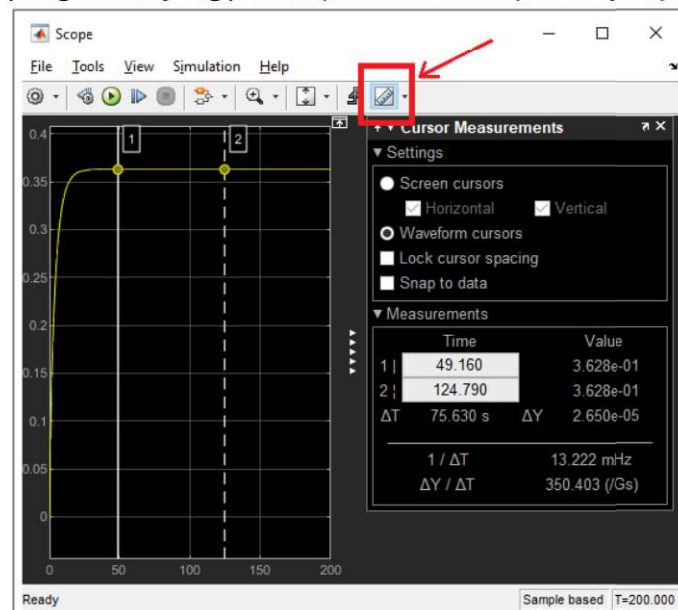
## Langkah-langkah Percobaan

1. **Buatlah model tangki menggunakan MATLAB/Simulink.** Contoh model Simulink diperlihatkan pada Gambar 2. Gunakan nilai-nilai berikut:
  - $A_i = 0.2 \text{ m}^2$
  - $A_o = 0.3 \text{ m}^2$
  - $V_i = 8 \text{ m/s}$
  - $A = 5 \text{ m}^2$
2. **Simpan model tersebut.** Simpan dengan nama tertentu (tanpa spasi), misalnya **modelTangki.slx**.



**Gambar 2. Pemodelan tangki sederhana pada MATLAB/Simulink**

3. Jalankan simulasi dan ubah-ubahlah parameter tangki. Seting **Stop Time** menjadi 200, dan selanjutnya jalankan simulasi (**Run**). Kemudian, ubah-ubah bukaan valve input (Ai) dan jalankan kembali simulasi. Gunakan **Cursor Measurement** pada **Scope** untuk melihat nilai pengukuran yang presisi (lihat Gambar 3). Selanjutnya, lengkapi Tabel 2.



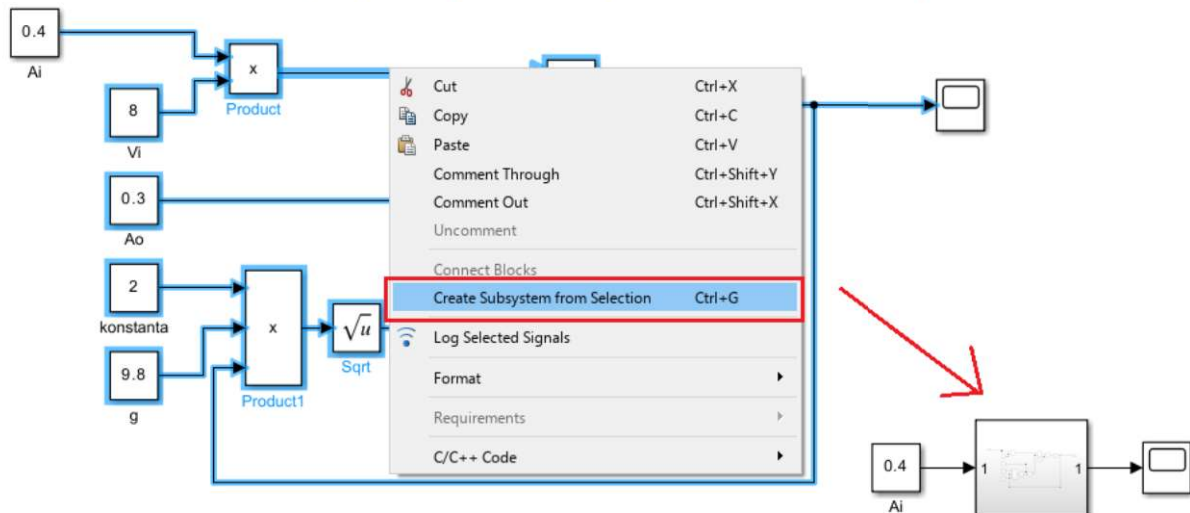
**Gambar 3. Penggunaan Cursor Measurement pada Scope**

Tabel 2. Pengamatan level air menggunakan simulasi Simulink

| No | Parameter | Nilai | Level Air (saat steady) | Waktu mencapai steady |
|----|-----------|-------|-------------------------|-----------------------|
| 1  | Ai        | 0.10  | ...                     | ...                   |
| 2  | Ai        | 0.15  | ...                     | ...                   |
| 3  | Ai        | 0.20  | ...                     | ...                   |
| 4  | Ai        | 0.25  | ...                     | ...                   |
| 5  | Ai        | 0.30  | ...                     | ...                   |
| 6  | Ai        | 0.35  | ...                     | ...                   |
| 7  | Ai        | 0.40  | ...                     | ...                   |

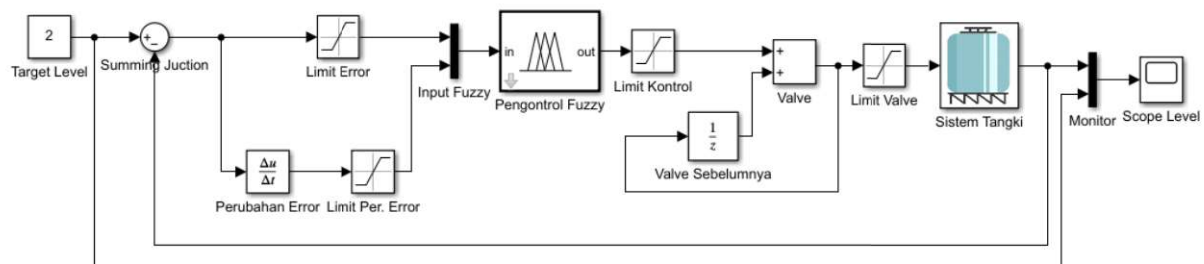


4. **Save As.. model tersebut.** Sekarang simpan simulasi dengan nama yang berbeda, misalnya **modelTangkiSubSistem.slx**
5. **Buat subsistem tangki.** Sekarang buatlah model tangki menjadi sebuah subsistem dengan mengeluarkan **Ai** dan **scope** (lihat Gambar 4). Anda dapat menambahkan gambar/logo pada subsistem yang telah dibuat dengan cara: **klik kanan subsistem → Mask → Add Icon Image**. Kemudian pilih logo yang diinginkan. Ingat, gambar/logo harus diletakkan di lingkungan kerja MATLAB (...\\Documents\\MATLAB).



**Gambar 4. Menjadikan sistem tangki menjadi subsistem**

6. **Buat loop tertutup untuk suatu sistem kontrol.** Kita akan mendesain kontrol kalang tertutup (*closed-loop*) seperti pada Gambar 5. Untuk membuat model Simulink seperti pada Gambar 5, **klik Library Model Browser**. Tambahkan semua komponen seperti pada Tabel 3. Perhatikan nama setiap komponen pada Gambar 5 dan cocokkan dengan Tabel 3. Untuk mencari tiap komponen, cari di kolom pencarian dengan mengetik nama jenis komponen (lihat kolom **Jenis Komponen** pada Tabel 3). Set nilai parameter setiap komponen dengan melakukan klik dua kali pada komponen, dan masukkan sesuai dengan ketentuan (lihat kolom **Nilai Parameter** pada Tabel 3). Selanjutnya, ikuti sambungan tiap sinyal sesuai dengan Gambar 5.



**Gambar 5. Model sistem kontrol tangki**

Tabel 3. Jenis komponen untuk pemodelan sistem tangki

| Nama Komponen    | Jenis Komponen                | Nilai Parameter         | Keterangan/Kegunaan                                            |
|------------------|-------------------------------|-------------------------|----------------------------------------------------------------|
| Target Level     | <b>Constant</b>               | 2                       | Target (set point) yang diinginkan                             |
| Summing Junction | <b>Sum</b>                    | +/-                     | Ganti dari ++ menjadi +-                                       |
| Limit Error      | <b>Saturation</b>             | Upper limit = 1.5       | Limit error dibuat sesuai dengan desain fuzzy                  |
|                  |                               | Lower limit = -1.5      |                                                                |
| Perubahan Error  | <b>Derivative</b>             | -                       | Melihat sebesar apa error berubah (kecepatan/derivative error) |
| Limit Per. Error | <b>Saturation</b>             | Upper limit = 0.004     | Limit perubahan error dibuat sesuai dengan desain fuzzy        |
|                  |                               | Lower limit = -0.004    |                                                                |
| Input Fuzzy      | <b>Mux</b>                    | -                       | Input fuzzy sebanyak 2                                         |
| Pengontrol Fuzzy | <b>Fuzzy Logic Controller</b> | (ditentukan kemudian)   | Nama fuzzy dimasukkan sesuai dengan nama desain                |
| Limit Kontrol    | <b>Saturation</b>             | Upper limit = 8e-05     | Limit bukaan valve dibuat sesuai dengan desain fuzzy           |
|                  |                               | Lower limit = -8e-05    |                                                                |
| Valve Sebelumnya | <b>UnitDelay</b>              | Initial Condition: 5e-3 | Kondisi valve saat ini bergantung dari posisi valve sebelumnya |
| Valve            | <b>Add</b>                    | -                       | Bukaan valve saat ini                                          |
| Limit Valve      | <b>Saturation</b>             | Upper limit = 7e-3      | Valve tidak boleh dibuka melewati batas desain                 |
|                  |                               | Lower limit = 0         |                                                                |
| Monitor          | <b>Mux</b>                    | -                       | Bandingkan output dengan input/target                          |
| Scope Level      | <b>Scope</b>                  | -                       | Memonitor sinyal                                               |

7. **Desain Kontrol Fuzzy.** Buka **Fuzzy Logic Designer**. Buat desain fuzzy dua input dan satu output. Berikut keterangannya:

**Input Fuzzy 1** → **error** [-1.5 1.5]

- **N:** Trapezoidal [nilai default]
- **Z:** Triangular [nilai default]
- **P:** Trapezoidal [nilai default]
- Kemudian klik **Evently Distribute MFs**

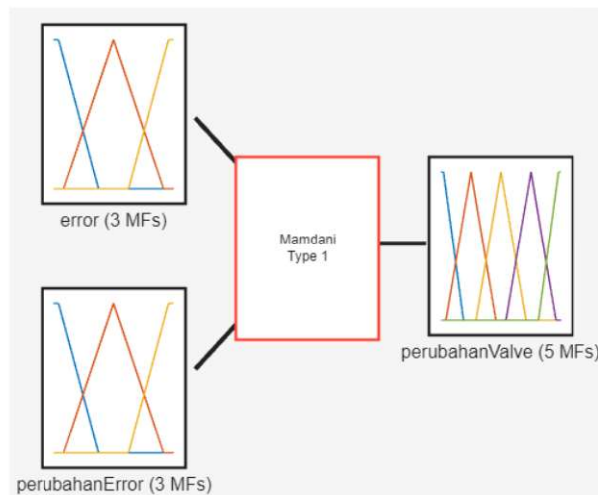
**Input Fuzzy 2** → **perubahanError** [-0.004 0.004]

- **N:** Trapezoidal [nilai default]
- **Z:** Triangular [nilai default]
- **P:** Trapezoidal [nilai default]
- Kemudian klik **Evently Distribute MFs**

**Output Fuzzy** → **perubahanValve** [-8e-05 8e-05]

- **NB:** Trapezoidal [nilai default]
- **NS:** Triangular [nilai default]
- **Z:** Triangular [nilai default]
- **PS:** Triangular [nilai default]
- **PB:** Trapezoidal [nilai default]
- Kemudian klik **Evently Distribute MFs**

Desain fuzzy akan terlihat seperti pada Gambar 6.



**Gambar 6. Desain kontroler awal**

8. **Tetapkan aturan fuzzy.** Buat aturan fuzzy dengan konektor **AND** sesuai dengan Gambar 7.

|       |   | perubahanError |    |    |
|-------|---|----------------|----|----|
|       |   | N              | Z  | P  |
| Error | N | Z              | NS | NB |
|       | Z | PS             | Z  | NS |
|       | P | PB             | PS | Z  |

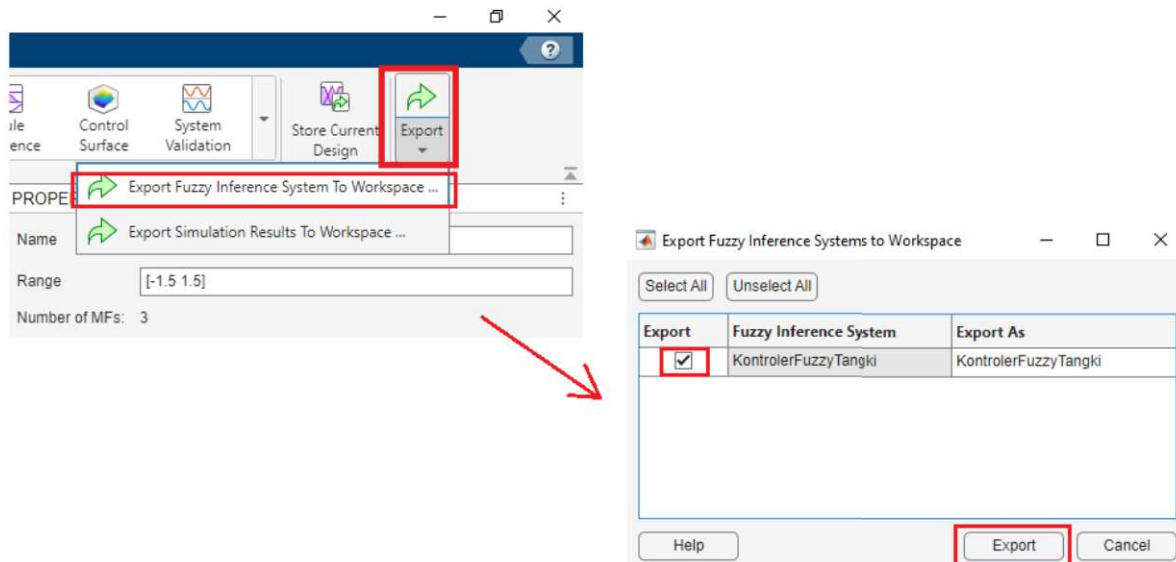
**Gambar 7. Fuzzy Rules**

**Keterangan:**

Z = Zero  
 P = Positive  
 N = Negative  
 PS = Positive Small  
 PB = Positive Big  
 NS = Negative Small  
 NB = Negatif Big

9. **Simpan desain.** Simpan desain pengontrol fuzzy. Misal, beri nama **KontrolerFuzzyTangki.fis**.
10. **Export desain ke workspace.** Sekarang, kita ekspor desain pengontrol fuzzy ke workspace. Pada menu **Export**, pilih **Export Fuzzy Inference System to Workspace ...**. Selanjutnya, cetang model dan klik **Export** (lihat Gambar 8).



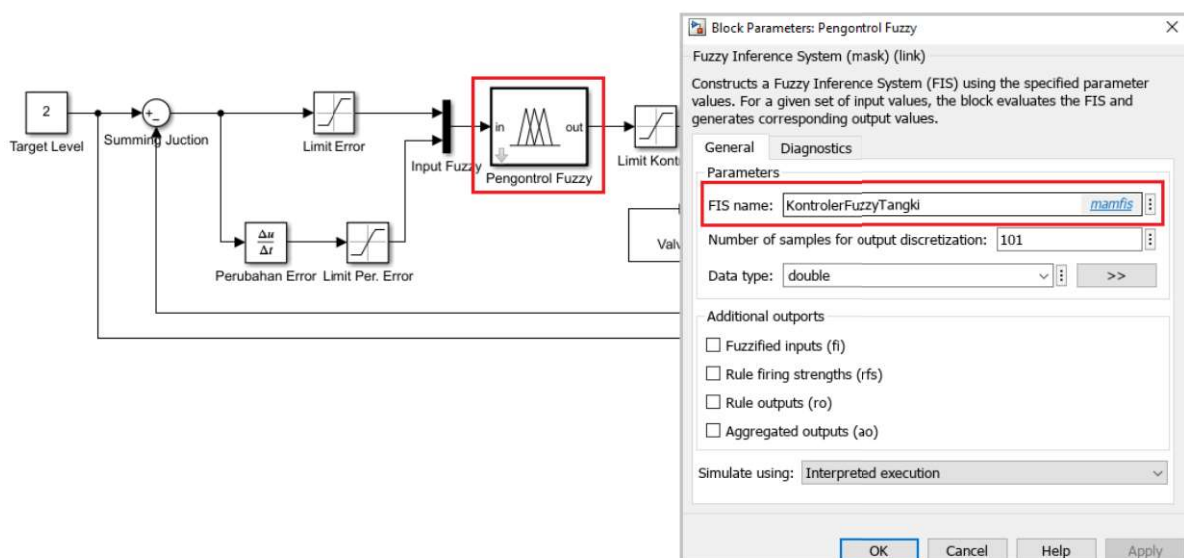


**Gambar 8. Ekspor model pengontrol fuzzy ke workspace**

11. **Kembali ke Simulink.** Kita edit sedikit parameter subsistem tangki sebagai berikut:

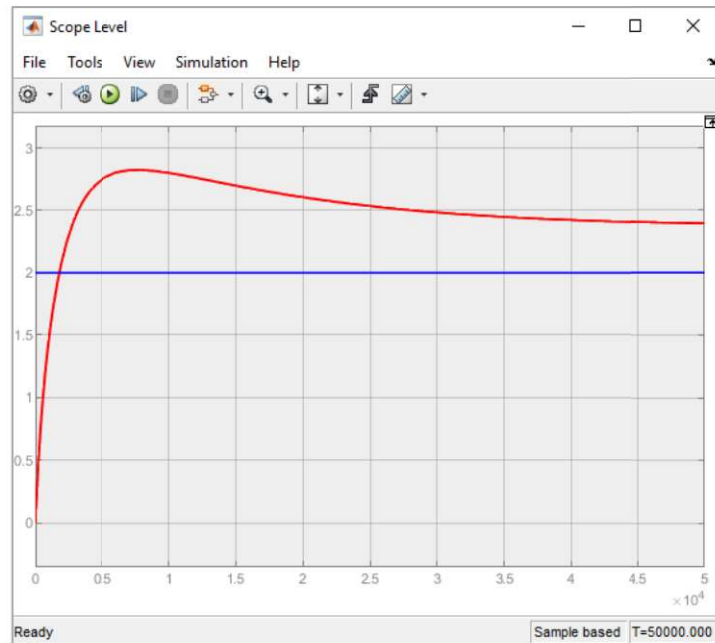
- **$V_i = 3$**
- **$A_o = 2e-3$**
- **$A = 4.91$**

12. **Masih di Simulink.** Klik 2x komponen **Pengontrol Fuzzy**, kemudian isi nama desain fuzzy kita, yaitu **KontrolerFuzzyTangki**, pada kolom **FIS name** (Lihat Gambar 9).



**Gambar 9. Menyertakan desain fuzzy ke Simulink**

13. **Menjalankan simulasi.** Seting **Stop Time** menjadi **50000**, kemudian klik logo **Run**. Setelah selesai, buka **Scope**. Amati hasilnya. Contoh respon output level air diperlihatkan pada Gambar 10.



**Gambar 10. Respon sistem berdasarkan pengontrol fuzzy**

## **Tugas Peserta Praktikum**

### **Tugas 1**

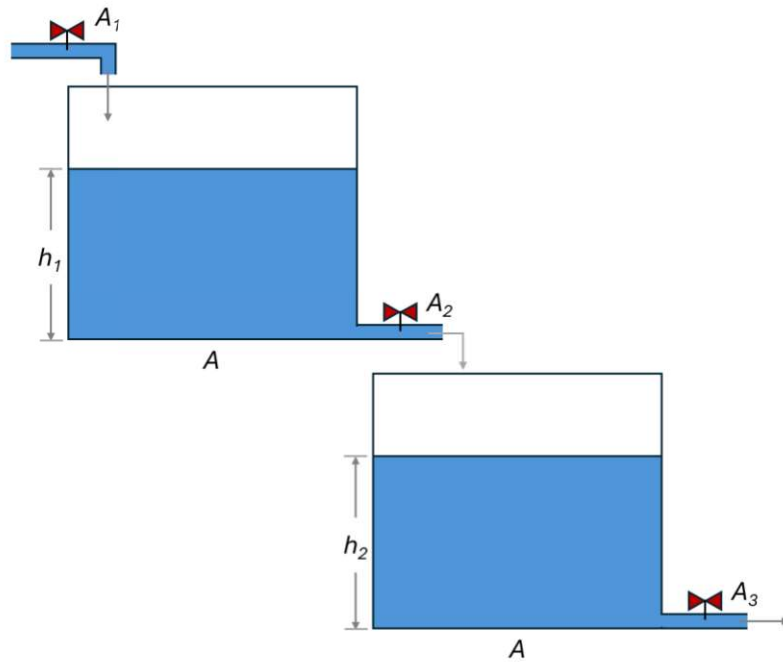
- Lengkapilah Tabel 2. Isi nilai yang diperlihatkan oleh Scope. Gunakan bantuan Cursor Measurement untuk mendapatkan nilai yang akurat

### **Tugas 2**

- Amatilah Gambar 10. Terlihat bahwa respon sistem belumlah cukup akurat untuk membawa level air sesuai dengan target yang diinginkan. Sekarang, optimalkan desain fuzzy Anda. Optimalisasi dapat dilakukan dengan berbagai cara:
  - Menggeser-geser membership function (MF) baik untuk kedua Input maupun Output sesuai dengan logika kontrol Anda.
  - Mengubah jenis membership function
  - Mengurangi/menambah membership function
  - Mengedit, menambah atau mengurangi rules

### **Tugas 3**

- Amatilah Gambar 11. Modelkanlah sistem dua tangki tersebut untuk melihat dinamika ketinggian cairan di tangki 1 ( $h_1$ ) dan tangki 2 ( $h_2$ ). Turunkan rumusnya dan modelkan pada Simulink. Asumsikan luas kedua penampang tangki adalah sama. Saat ini, kita tidak perlu merancang sistem kontrolnya.



**Gambar 11. Sistem dua tangki**



## MODUL 3

# Penerapan Fuzzy Logic pada Mikrokontroler untuk Sistem Kontrol Cerdas Berbasis Embedded System

### Pendahuluan

Perkembangan sistem kendali modern tidak hanya berhenti pada simulasi di MATLAB atau Simulink, tetapi juga mengarah pada implementasi langsung di perangkat tertanam seperti mikrokontroler. Dalam praktikum ini, mahasiswa akan mempelajari bagaimana logika fuzzy diprogram dan dijalankan pada **mikrokontroler** sebagai dasar penerapan kontrol cerdas pada perangkat berukuran kecil, murah, dan hemat daya.

Implementasi fuzzy pada mikrokontroler sangat bermanfaat dalam berbagai bidang, terutama Internet of Things (IoT), edge intelligence, dan Artificial Intelligence of Things (AloT). Dengan pendekatan ini, proses pengambilan keputusan tidak selalu harus dikirim ke komputer atau server, tetapi dapat dilakukan langsung di sisi perangkat. Hal ini membuat sistem menjadi lebih cepat merespons, lebih hemat komunikasi data, dan lebih sesuai untuk aplikasi nyata seperti kontrol suhu, kelembapan, level air, kecepatan motor, irigasi otomatis, monitoring lingkungan, serta sistem otomasi sederhana lainnya.

Melalui praktikum ini, mahasiswa akan memahami bahwa kontrol fuzzy dapat diterapkan tidak hanya secara konseptual, tetapi juga secara praktis pada perangkat fisik. Mahasiswa akan belajar menerjemahkan membership function, rule base, dan proses inferensi fuzzy ke dalam bentuk program yang dapat dijalankan oleh mikrokontroler. Dengan demikian, praktikum ini menjadi jembatan antara konsep kontrol cerdas, pemrograman embedded system, dan penerapan teknologi AloT untuk sistem kendali.

### Tujuan

Setelah menyelesaikan praktikum ini, Mahasiswa diharapkan mampu:

1. Memahami konsep implementasi sistem fuzzy pada mikrokontroler sebagai bagian dari kontrol cerdas berbasis embedded system.
2. Menerjemahkan membership function dan rule base fuzzy ke dalam bentuk program yang dapat dijalankan pada mikrokontroler.
3. Melakukan proses inferensi fuzzy secara langsung pada mikrokontroler berdasarkan data input dari sensor atau simulasi input.
4. Menganalisis potensi penerapan fuzzy berbasis mikrokontroler untuk sistem IoT, edge intelligence, dan AloT dalam bidang kontrol.

## Alat dan Bahan

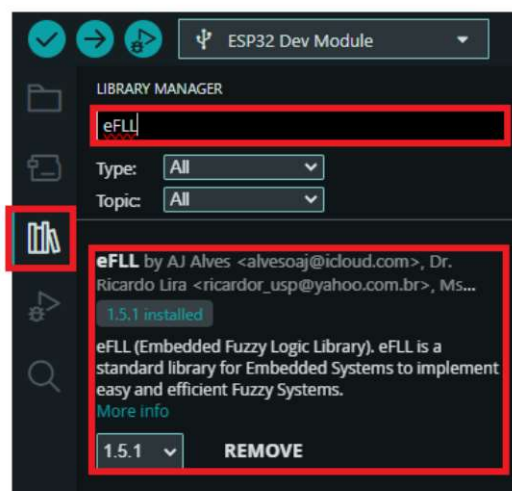
Alat yang dibutuhkan diperlihatkan pada Tabel 1.

Tabel 1. Kebutuhan praktikum

| No | Nama Alat dan Bahan         | Jumlah | Keterangan                |
|----|-----------------------------|--------|---------------------------|
| 1  | Mikrokontroler              | 1      | Perangkat terprogram      |
| 2  | MATLAB/Fuzzy Logic Designer | 1      | Verifikasi desain fuzzy   |
| 3  | Arduino IDE                 | 1      | Platform untuk memprogram |

## Langkah-langkah Percobaan

1. **Buka Arduino IDE.** Install pustaka yang diperlukan, yaitu **eFLL** by AJ Alves. Perhatikan Gambar 1.



Gambar 1. Instalasi pustaka fuzzy

2. **Studi Kasus.** Kita kembali akan menggunakan studi kasus “**Dinner for Two**”. Pelajari kembali modul 1 praktikum untuk mengingat kasus ini.
3. **Ketik skrip program.** Ketiklah skrip program berikut secara bertahap. Di saat yang sama, tolong belajar untuk dipahami.

Panggil pustaka pada skrip kita:

```
#include <Fuzzy.h>
```

Deklarasikan sebuah variabel penunjuk (*pointer*) bernama **fuzzy** yang menunjuk ke objek dari kelas (class) bernama **Fuzzy**.

```
Fuzzy *fuzzy = new Fuzzy();
```

Deklarasi dan Instansiasi Fungsi Keanggotaan (Membership Functions). Pertama, untuk tulis untuk **Pelayanan**.

```
// FuzzyInput  
FuzzySet *jelek = new FuzzySet(0, 0, 0, 4);
```



```
FuzzySet *sedang = new FuzzySet(1, 5, 5, 9);
FuzzySet *bagus = new FuzzySet(6, 10, 10, 10);
```

Lanjutkan ke input **Makanan**.

```
// FuzzyInput
FuzzySet *tengik = new FuzzySet(0, 0, 1, 3);
FuzzySet *lezat = new FuzzySet(7, 9, 10, 10);
```

Setelah selesai input, maka lanjutkan ke output, yaitu **Tip**.

```
// FuzzyOutput
FuzzySet *murah = new FuzzySet(0, 5, 5, 10);
FuzzySet *standar = new FuzzySet(10, 15, 15, 20);
FuzzySet *mahal = new FuzzySet(20, 25, 25, 30);
```

Sekarang masuk ke void setup. Inisialisasi kecepatan serial untuk menampilkan hasil fuzzy ke Serial Monitor.

```
void setup()
{
    Serial.begin(9600);
}
```

Semua membership function (MF) telah dideklarasikan. Namun, program belum tahu yang mana MF milik input dan yang mana milik output. Program belum tahu apa nama variabel input dan variabel output. Sekarang, *assign* (tugaskan) tiap MF ke input/output dan beri indeks untuk input dan output. Pertama, assign untuk Pelayanan. Beri nama dan indeks.

```
// FuzzyInputPelayanan
FuzzyInput *pelayanan = new FuzzyInput(1);
pelayanan->addFuzzySet(jelek);
pelayanan->addFuzzySet(sedang);
pelayanan->addFuzzySet(bagus);
fuzzy->addFuzzyInput(pelayanan);
```

Lanjut ke input Makanan

```
// FuzzyInputMakanan
FuzzyInput *makanan = new FuzzyInput(2);
makanan->addFuzzySet(tengik);
makanan->addFuzzySet(lezat);
fuzzy->addFuzzyInput(makanan);
```



Lanjutkan lagi ke output

```
// FuzzyOutputTip
FuzzyOutput *tip = new FuzzyOutput(1);
tip->addFuzzySet(murah);
tip->addFuzzySet(standar);
tip->addFuzzySet(mahal);
fuzzy->addFuzzyOutput(tip);
```

Sekarang, susun aturannya (*rules*). Menyusun aturan terdiri dari 3 tahapan, yaitu

1. deklarasikan bagian **anteseden**,
2. deklarasikan bagian **konsekuen**, dan
3. **gabungkan** anteseden dan konsekuen.



Aturan 1: Jika **Pelayanan Jelek** ATAU **Makanan Tengik**, maka **Tip Murah**

```
// Building FuzzyRule 1
// Bagian anteseden
FuzzyRuleAntecedent *ifPelayananJelekORMakananTengik = new FuzzyRuleAntecedent();
ifPelayananJelekORMakananTengik->joinWithOR(jelek, tengik);

// Bagian konsekuen
FuzzyRuleConsequent *thenTipMurah = new FuzzyRuleConsequent();
thenTipMurah->addOutput(murah);

// Gabungkan anteseden dan konsekuen
FuzzyRule *fuzzyRule1 = new FuzzyRule(1, ifPelayananJelekORMakananTengik, thenTipMurah);
fuzzy->addFuzzyRule(fuzzyRule1);
```

Aturan 2: Jika **Pelayanan Sedang**, maka **Tip Standar**

```
// Building FuzzyRule 2
FuzzyRuleAntecedent *ifPelayananSedang = new FuzzyRuleAntecedent();
ifPelayananSedang->joinSingle(sedang);

FuzzyRuleConsequent *thenTipStandar = new FuzzyRuleConsequent();
thenTipStandar->addOutput(standar);

FuzzyRule *fuzzyRule2 = new FuzzyRule(2, ifPelayananSedang, thenTipStandar);
fuzzy->addFuzzyRule(fuzzyRule2);
```

### Aturan 3: Jika **Pelayanan Bagus** ATAU **Makanan Lezat**, maka **Tip Mahal**

```
// Building FuzzyRule 3

FuzzyRuleAntecedent *ifPelayananBagusORMakananLezat = new FuzzyRuleAntecedent();
ifPelayananBagusORMakananLezat->joinWithOR(bagus, lezat);

FuzzyRuleConsequent *thenTipMahal = new FuzzyRuleConsequent();
thenTipMahal->addOutput(mahal);

FuzzyRule *fuzzyRule3 = new FuzzyRule(3, ifPelayananBagusORMakananLezat, thenTipMahal);
fuzzy->addFuzzyRule(fuzzyRule3);
}
```

Sekarang masuk ke void loop. Di sini kita mulai memberikan nilai untuk setiap input. Pada kasus terkahir, kita gunakan Pelayanan = 7, dan Makanan = 8

```
void loop()
{
    int input1 = 7;
    int input2 = 8;

    fuzzy->setInput(1, input1);
    fuzzy->setInput(2, input2);
}
```

Sekarang, lakukan fuzzyfikasi

```
fuzzy->fuzzify();
```

Lakukan defuzzyfikasi kembali dan simpan hasilnya pada variabel dengan nama "output".

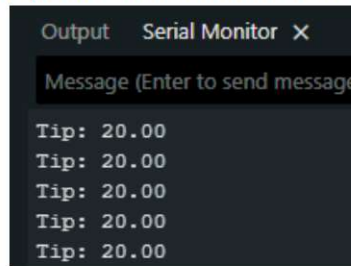
```
float output = fuzzy->defuzzify(1);
```

Terakhir, keluarkan hasil pada serial monitor dan tunggu beberapa detik, misalnya 10 detik.

```
Serial.print("Tip: ");
Serial.println(output);

// wait 12 seconds
delay(12000);
}
```

Amatilah hasil yang ditampilkan pada serial monitor berikut:



## Tugas Peserta Praktikum

### Tugas 1

- Amati kasus di bawah ini dan buatlah programnya pada Arduino.
- Verifikasi hasilnya menggunakan MATLAB

Suatu ruangan pertemuan menggunakan beberapa kipas untuk mengkondisikan udara dalam ruangan. Parameter input adalah **Suhu [20-40]** dan **Kelembapan [20-90]**. Parameter yang dikontrol adalah level **Kecepatan kipas [0-100%]**.

Berikut keterangan Input dan Output kontroler:

**Input 1: Suhu (°C): [20 40]**

- **dingin:** trapmf [20 20 24 26]
- **hangat:** trimf [25 28.5 32]
- **panas:** trapmf [31 34 40 40]

**Input 2: Kelembapan (%): [20 90]**

- **rendah:** trapmf [20 20 35 45]
- **sedang:** trimf [40 55 70]
- **tinggi:** trapmf [65 75 90 90]

**Output: KecepatanKipas (%): [0 100]**

- **rendah:** trapmf [0 0 20 40]
- **sedang:** trimf [30 50 70]
- **tinggi:** trapmf [60 80 100 100]

Rules ditentukan sebagai berikut:

|      |        | Kelembapan |        |        |
|------|--------|------------|--------|--------|
|      |        | Rendah     | Sedang | Tinggi |
| Suhu | Dingin | Rendah     | Rendah | Sedang |
|      | Hangat | Rendah     | Sedang | Tinggi |
|      | Panas  | Sedang     | Tinggi | Tinggi |



## Tugas 2

- Amati kasus di bawah ini dan buatlah programnya pada Arduino.
- Verifikasi hasilnya menggunakan MATLAB

Sebuah sistem AC otomatis dirancang untuk mengatur **daya pendingin** berdasarkan **suhu ruangan** dan **jumlah orang** di dalam ruangan. Semakin tinggi suhu dan semakin banyak orang, maka daya AC yang dibutuhkan juga semakin besar. Sebaliknya, jika suhu ruangan rendah dan jumlah orang sedikit, daya AC dapat dibuat lebih rendah agar lebih hemat energi. Dengan logika fuzzy, kondisi suhu dan jumlah orang dibagi ke dalam beberapa kategori untuk menentukan keluaran berupa **daya AC** secara lebih fleksibel dan adaptif. Berikut keterangan Input dan Output kontroler:

**Input 1: SuhuRuangan (°C): [18 35]**

- **rendah:** trapmf [18 18 20 23]
- **sedang:** trimf [22 26 30]
- **tinggi:** trapmf [28 32 35 35]

**Input 2: JumlahOrang: [0 10]**

- **sedikit:** trapmf [0 0 2 4]
- **sedang:** trimf [3 5 7]
- **banyak:** trapmf [6 8 10 10]

**Output: DayaAC (%): [0 100]**

- **rendah:** trapmf [0 0 20 40]
- **sedang:** trimf [30 50 70]
- **tinggi:** trapmf [60 80 100 100]

Rules ditentukan sebagai berikut (gunakan konektor **AND**):

|             |        | JumlahOrang |        |        |
|-------------|--------|-------------|--------|--------|
|             |        | Sedikit     | Sedang | Banyak |
| SuhuRuangan | Rendah | Rendah      | Rendah | Sedang |
|             | Sedang | Sedang      | Sedang | Tinggi |
|             | Tinggi | Tinggi      | Tinggi | Tinggi |

## MODUL 4

# Implementasi dan Analisis Kontrol Fuzzy Motor DC Menggunakan MATLAB dan Mikrokontroler

### Pendahuluan

Motor DC merupakan salah satu aktuator yang banyak digunakan dalam sistem industri, robotika, otomasi, kendaraan listrik skala kecil, conveyor, pompa, kipas, dan berbagai perangkat mekatronika. Dalam penerapannya, motor tidak cukup hanya dinyalakan atau dimatikan, tetapi perlu dikendalikan agar dapat bekerja sesuai kebutuhan sistem. Salah satu parameter penting yang sering dikontrol adalah kecepatan putaran motor. Kecepatan motor dapat berubah akibat variasi beban, perubahan tegangan, gesekan mekanik, atau kondisi kerja lainnya, sehingga diperlukan suatu metode kontrol yang mampu menjaga performa motor agar tetap stabil.

Pada praktikum ini, mahasiswa akan mempelajari penerapan kontrol fuzzy untuk mengatur kecepatan motor DC menggunakan mikrokontroler. Mikrokontroler digunakan sebagai pusat pengendali yang membaca kecepatan motor melalui encoder, menghitung error antara setpoint dan kecepatan aktual, melakukan proses inferensi fuzzy, lalu menghasilkan sinyal PWM untuk mengatur daya motor. Praktikum ini memberikan pengalaman langsung kepada mahasiswa dalam menghubungkan konsep kontrol cerdas dengan implementasi nyata pada perangkat embedded. Melalui kegiatan ini, mahasiswa diharapkan memahami bahwa kontrol fuzzy tidak hanya dapat disimulasikan, tetapi juga dapat diterapkan secara praktis pada sistem aktuator di dunia nyata.

### Tujuan

Setelah menyelesaikan praktikum ini, Mahasiswa diharapkan mampu:

1. Memahami prinsip dasar pengendalian kecepatan motor DC menggunakan mikrokontroler.
2. Merancang sistem kontrol fuzzy sederhana dengan masukan berupa error dan delta error, serta keluaran berupa perubahan nilai PWM.
3. Mengimplementasikan algoritma kontrol fuzzy pada mikrokontroler untuk mengatur kecepatan motor DC secara real-time.
4. Mengamati respons kecepatan motor terhadap perubahan setpoint dan mengevaluasi kinerja kontrol fuzzy yang digunakan.

### Alat dan Bahan

Alat yang dibutuhkan diperlihatkan pada Tabel 1.

Tabel 1. Kebutuhan praktikum

| No | Nama Alat dan Bahan | Jumlah | Keterangan           |
|----|---------------------|--------|----------------------|
| 1  | Mikrokontroler      | 1      | Perangkat terprogram |
| 2  | Driver L298N        | 1      | Driver motor         |

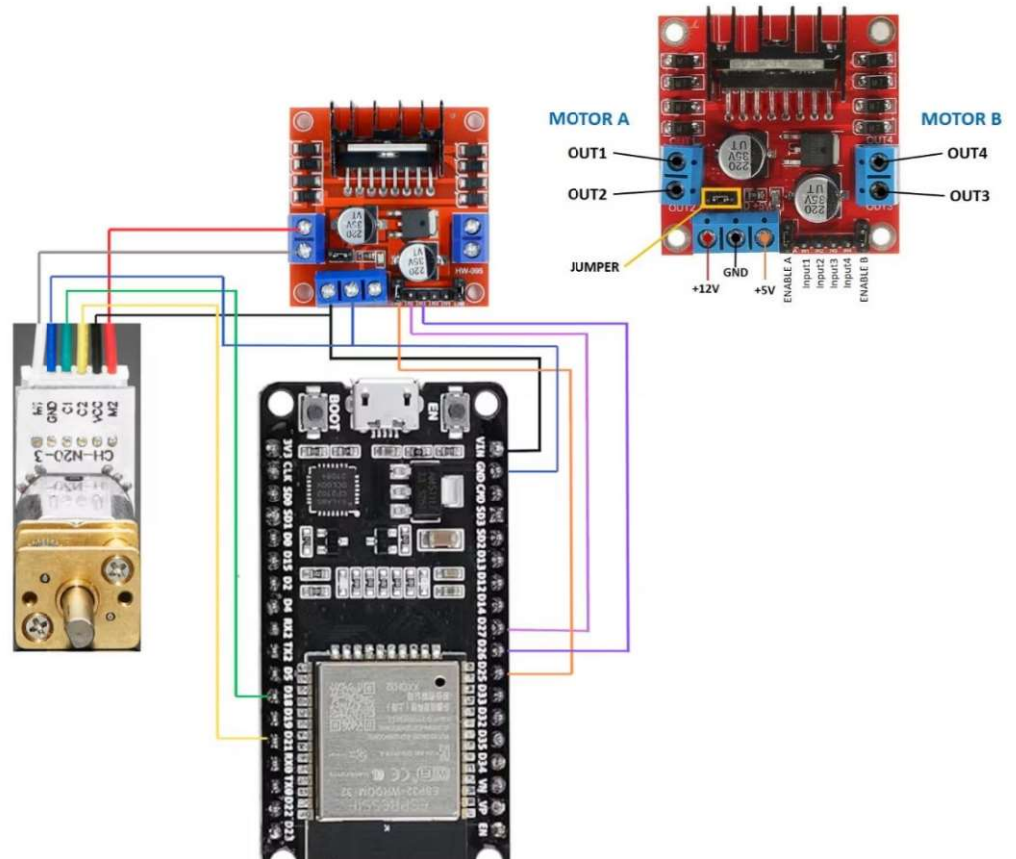


|   |                             |   |                           |
|---|-----------------------------|---|---------------------------|
| 3 | MATLAB/Fuzzy Logic Designer | 1 | Verifikasi desain fuzzy   |
| 4 | Arduino IDE                 | 1 | Platform untuk memprogram |

## Langkah-langkah Percobaan

### TAHAP 1 – Implementasi Kontrol Closed-loop pada Mikrokontroler

1. **Rakit hardware.** Hubungkan motor DC, driver motor dan ESP32 berdasarkan skematik pada Gambar 1.



**Gambar 1. Rangkaian motor kontrol**

Tabel koneksi antara mikrokontroler, motor dan driver diperlihatkan pada Tabel 2. **Pastikan mikrokontroler, motor dan driver telah terkoneksi dengan benar.**

**Tabel 2. Koneksi antar modul**

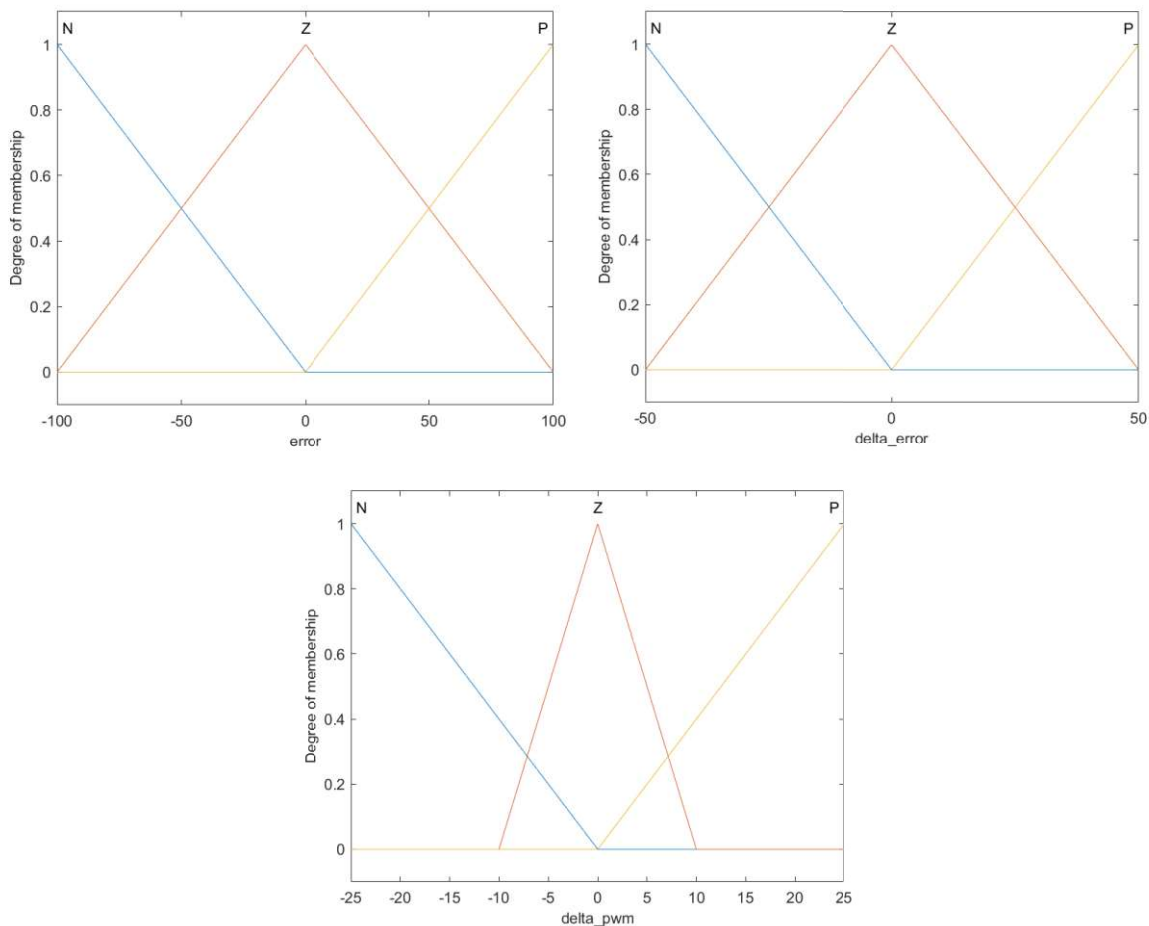
| Motor DC | ESP32 | Driver L298N           |
|----------|-------|------------------------|
| M2       | -     | OUT1                   |
| VCC      | VIN   | +12V                   |
| C2       | D21   | -                      |
| C1       | D18   | -                      |
| GND      | GND   | GND                    |
| M1       | -     | OUT2                   |
| -        | D25   | ENA (cabut jumper ENA) |
| -        | D26   | IN2                    |
| -        | D27   | IN1                    |



2. **Desain Kontrol Fuzzy.** Kontrol fuzzy didesain seperti berikut:

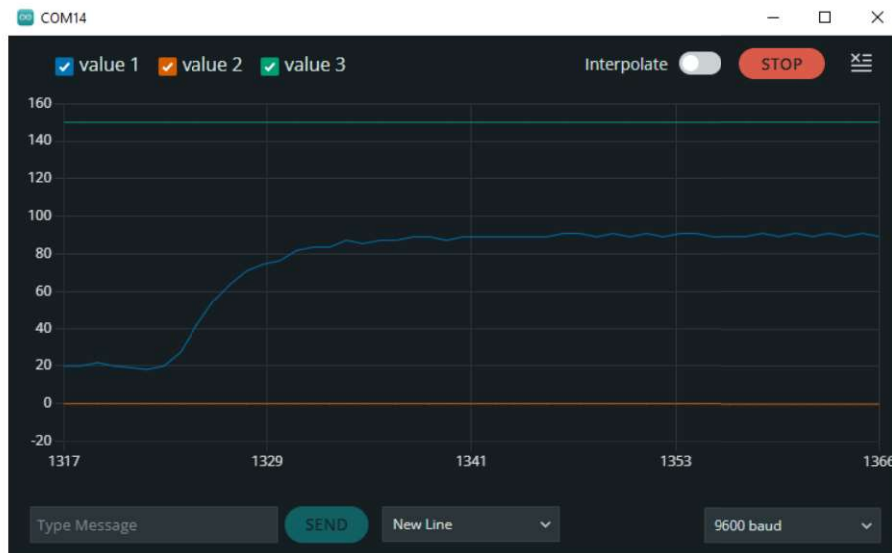
| Input 1: <b>error</b> [-100 100]                                                                                                                      | Input 2: <b>delta_error</b> [-50 50]                                                                                                           | Output: <b>delta_pwm</b> [-25 25]                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• N: trapmf [-100 -100 100 0]</li> <li>• Z: trimf [-100 0 100]</li> <li>• P: trapmf [0 100 100 100]</li> </ul> | <ul style="list-style-type: none"> <li>• N: trapmf [-50 -50 -50 0]</li> <li>• Z: trimf [-50 0 50]</li> <li>• P: trapmf [0 50 50 50]</li> </ul> | <ul style="list-style-type: none"> <li>• N: trapmf [-25 -25 -25 0]</li> <li>• Z: trimf [-10 0 10]</li> <li>• P: trapmf [0 25 25 25]</li> </ul> |

Secara visual, membership function (MF) untuk input dan output dapat digambarkan sebagai berikut:



**Gambar 2. Membership Function untuk input dan output fuzzy**

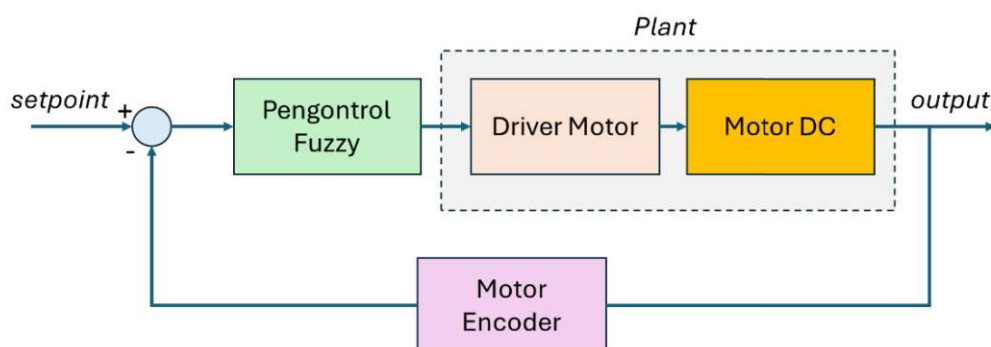
3. **Eksekusi Skrip Program Arduino.** Unduh program untuk mengontrol motor DC di <https://github.com/kusuma86/KONTROL-CERDAS-2026/blob/main/LABS/fuzzyArduino.ino>. Buka Arduino dan upload skrip program tersebut ke mikrokontroler.
4. **Jalankan Serial Plotter.** Bukalah Serial Plotter, seting kecepatan serial (baud rate), dan masukkan nilai RPM yang diinginkan. Direkomendasikan untuk memasukkan nilai antara **10 – 120** di Serial Plotter. **Gambar 3** menampilkan contoh bagaimana motor berusaha mencapai target, yaitu sebesar 90 RPM.



**Gambar 3. Serial Plotter untuk mengontrol kecepatan motor DC**

**Amati koding Arduino.** Perhatikan bahwa sistem kontrol kalang tertutup (*closed-loop*) secara langsung diimplementasikan pada mikrokontroler. Carilah potongan program Arduino berikut dan pahami maksudnya:

| Potongan Program                                                                                                                                                                                                          | Keterangan                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// --- 3. Hitung error &amp; delta_error --- error = setpoint - actual_rpm; delta_error = error - last_error;</pre>                                                                                                  | Mikrokontroler menghitung nilai error dan perubahan/kecepatan error dari motor saat ini. Kedua hasil perhitungan ini menjadi input kontroler fuzzy                                                                                                                                                                  |
| <pre>// --- 4. Fuzzy Logic Inferensi --- fuzzy-&gt;setInput(1, error); fuzzy-&gt;setInput(2, delta_error); fuzzy-&gt;fuzzify(); float delta_pwm = fuzzy-&gt;defuzzify(1);</pre>                                           | Error dan delta_error menjadi input untuk fuzzy kita. Langsung melakukan proses fuzzifikasi dan defuzzifikasi untuk mendapatkan output dari fuzzy                                                                                                                                                                   |
| <pre>// --- 5. Update PWM &amp; kendali motor --- pwmValue += delta_pwm; pwmValue = constrain(pwmValue, 0, 255);  digitalWrite(ForwardPin, HIGH); digitalWrite(BackwardPin, LOW); analogWrite(EnablePin, pwmValue);</pre> | Output fuzzy adalah <b>perubahan PWM</b> . Jadi, nilai yang diberikan oleh fuzzy kepada driver motor adalah nilai <b>output fuzzy saat ini + nilai sebelumnya</b> . Ini sama analoginya dengan membuka kran air. Posisi kran air saat ini adalah hasil dari posisi sebelumnya, ditambah dengan nilai aksi saat ini. |

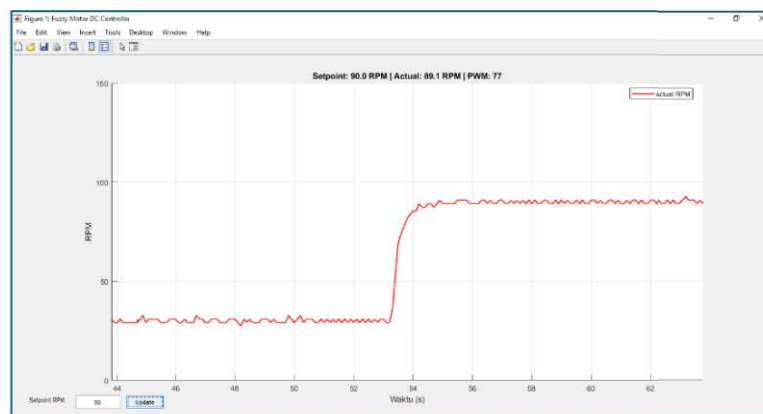


**Gambar 4. Sistem kontrol kalang tertutup untuk motor DC**

5. **Ambil beberapa Screenshot.** Ambillah beberapa gambar plotting dengan setpoint yang berbeda sebagai bahan laporan.

## TAHAP 2 – Memindahkan Pengontrol Fuzzy ke MATLAB

1. **Lanjut ke memindahkan kontroler ke MATLAB.** Setelah sukses menerapkan sistem kalang tertutup pada mikrokontroler, kegiatan selanjutnya adalah menghubungkan mikrokontroler dan MATLAB sebagai salah satu skema kontrol. **Kontrol fuzzy akan kita pindahkan ke MATLAB.** Mikrokontroler hanya membaca nilai RPM aktual saat ini dan mengirimkan nilai tersebut ke MATLAB melalui komunikasi serial.
2. **Program untuk Arduino.** Unduh dan unggah program berikut ke mikrokontroler <https://github.com/kusuma86/KONTROL-CERDAS-2026/blob/main/LABS/kirimRPMSerial.ino>
3. **Program untuk MATLAB.** Unduhlah program MATLAB berikut dan letakkan pada folder direktori aktif MATLAB Anda (biasanya di Documents\MATLAB\)  
<https://github.com/kusuma86/KONTROL-CERDAS-2026/blob/main/LABS/kontrolFuzzyMotor.m>
4. **Jalankan Program MATLAB.** Pastikan komunikasi Serial pada Arduino IDE tertutup (tutup Serial Monitor atau Serial Plotter).
5. Isi set point awal dan selanjutnya kontrol menggunakan GUI seperti berikut:



**Gambar 4. Antarmuka (GUI) untuk kontrol motor DC pada MATLAB**

6. **Ambil beberapa Screenshot.** Ambillah beberapa gambar hasil plotting dengan setpoint yang berbeda sebagai bahan laporan.

## TAHAP 3 – Logger Data dan Analisis Respon Sistem

1. **Lanjut melakukan logger dan analisis respon sistem.** Setelah sukses memindahkan kontroler ke MATLAB, kita akan lanjut melakukan analisis terhadap respon sistem dari kontroler yang telah kita rancang. **Posisi kontroler fuzzy masih ada di MATLAB.** MATLAB akan melakukan logger data selama 20 detik, kemudian menampilkan hasil analisis terhadap respon sistem.
2. **Unduh program MATLAB.** Unduhlah program MATLAB berikut:  
<https://github.com/kusuma86/KONTROL-CERDAS-2026/blob/main/LABS/loggerDanAnalisisMATLAB.m>



3. **Catatlah hasil pengamatan.** Gunakan Tabel di bawah ini. Catat minimal 5 jenis setpoint yang berbeda.

**Setpoint = ....**

| Parameter               | Percobaan |   |   | Rata-rata |
|-------------------------|-----------|---|---|-----------|
|                         | 1         | 2 | 3 |           |
| RPM maksimum            |           |   |   |           |
| Overshoot (RPM)         |           |   |   |           |
| Overshoot (%)           |           |   |   |           |
| Rise Time               |           |   |   |           |
| Settling time $\pm 5\%$ |           |   |   |           |
| RPM steady-state        |           |   |   |           |
| Std RPM steady-state    |           |   |   |           |
| Steady-state error      |           |   |   |           |

4. **Gunakan AI.** Cari tahu apa pengertian dari istilah-istilah yang ditampilkan dalam analisis MATLAB, misalnya Overshoot, Rise Time, dsb.